



APRENDIZADO PROFUNDO

deep learning

PROF. JOSENALDE OLIVEIRA

PROGRAMA DE PÓS-GRADUAÇÃO EM TI (PPgTI) - UFRN

O benefício da profundidade...(hidden layers)

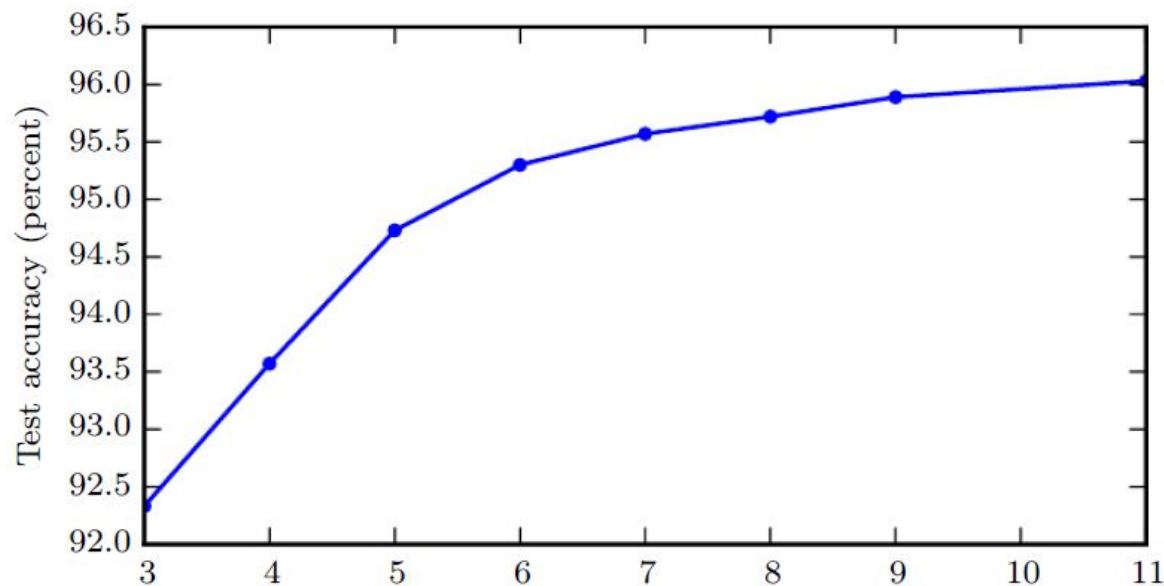
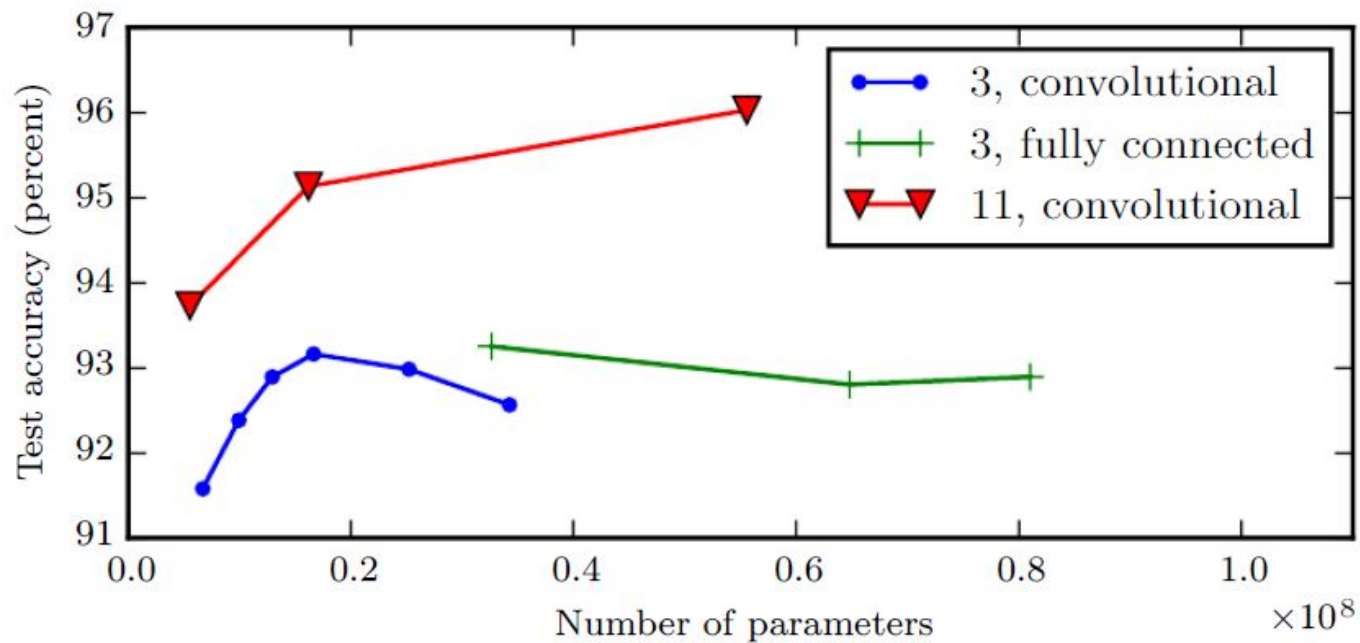


Figure 6.6: Empirical results showing that deeper networks generalize better when used to transcribe multi-digit numbers from photographs of addresses. Data from Goodfellow *et al.* (2014d). The test set accuracy consistently increases with increasing depth. See figure 6.7 for a control experiment demonstrating that other increases to the model size do not yield the same effect.

Fonte: Goodfellow et al. (2016)

- DNN tem uma eficiência de parâmetro bem maior que redes rasas, pois modelam funções complexas utilizando de forma exponencial menos neurônios
- **camadas inferiores** captam estruturas de **baixo nível** (segmentos de linhas de diversos formatos e orientações); as **intermediárias** combinam estas e modelam quadrados, círculos etc.; já as **superiores** e de saída combinam estas para modelar **alto nível**, como rostos de pessoas
- para problemas mais complexos pode-se aumentar as HL até começar a **sobreajustar**...

O benefício da profundidade...(hidden layers)



Fonte: Goodfellow et al. (2016)

- Aumentar número de parâmetros sem profundidade em rede densa - não melhora
- Aumentar número de parâmetros em rede CNN com 3 camadas - não melhora e sobreajusta
- Aumentar número de parâmetros e camadas (11) melhora!

O ponto entre o *under* e o *over* (fitting)

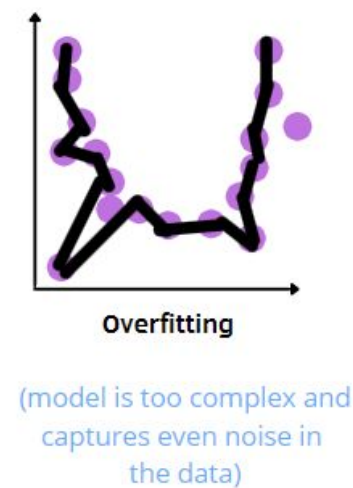
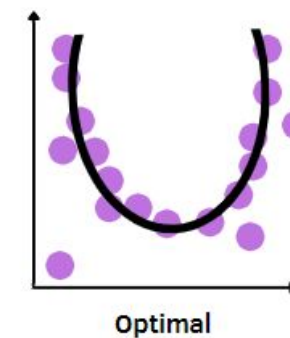
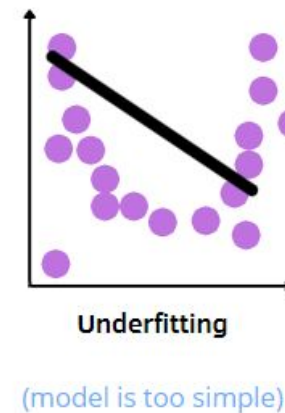
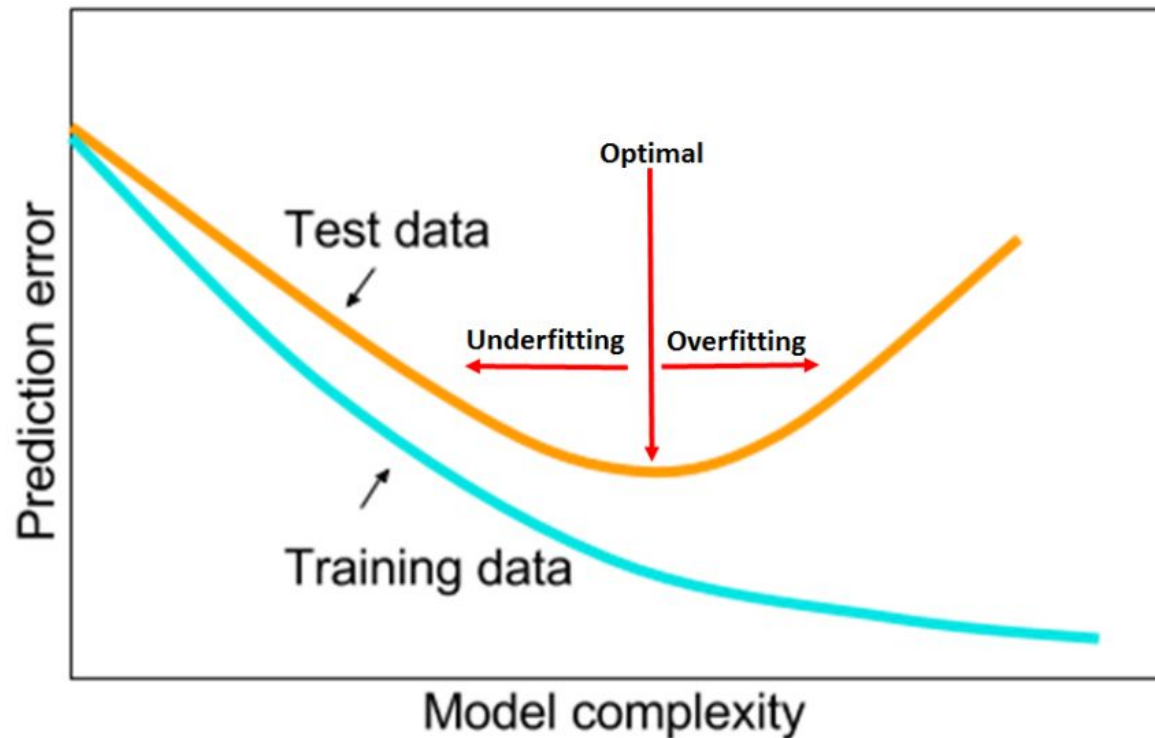
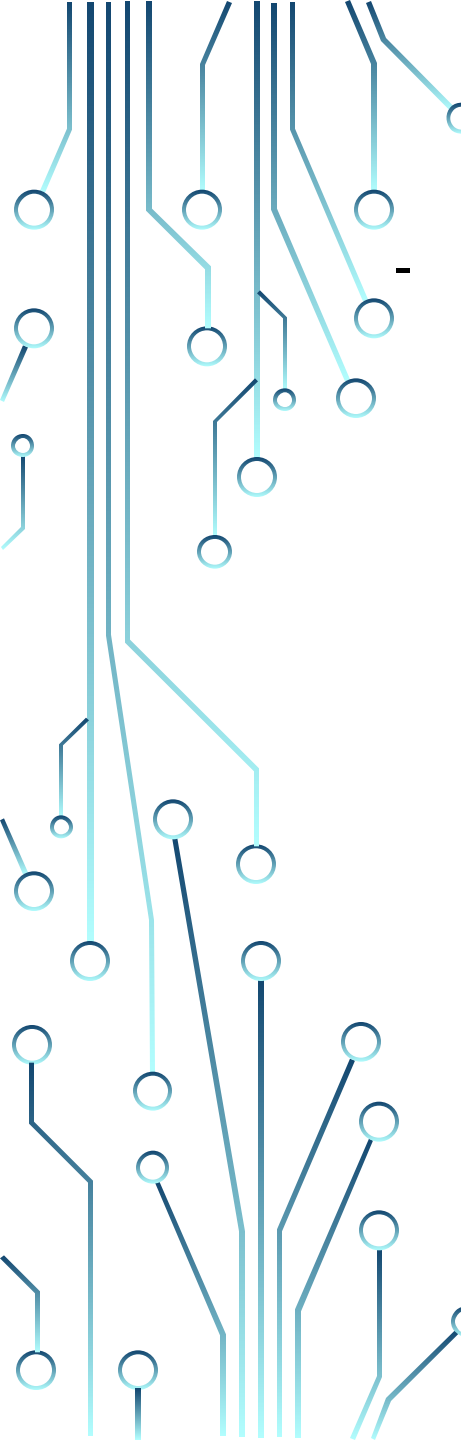


Figure 2: Pictorial explanation of the tradeoff between underfitting and overfitting. Model complexity (the x axis) refers to the capacity or powerfulness of the machine learning model. The figure shows the optimal capacity that falls between underfitting and overfitting.

- [Smith, L. \(2018\)](#)



O benefício da profundidade...(hidden layers)

- em geral tem-se uma relação custo-benefício maior aumentando o número de camadas em vez do número de neurônios por camada
 - há uma linha de pensar o número de neurônios como uma pirâmide, com cada vez menos neurônios por camada (características de baixo nível se aglutinam em menos características de alto nível). Por exemplo para o MNIST, pode ter 3 HL, 1) 300 units, 2) 200, 3) 100
 - outra linha usa o mesmo número nas camadas (para só ter um hiperparâmetro)
 - mas tem-se visto a primeira HL com mais neurônios
 - pode-se escolher modelo com mais camadas e neurônios do que de fato precisa e em seguida usar técnicas de *early stopping* e outras de regularização
 - O tamanho do batch pode impactar no desempenho e tempo de treinamento. Atualmente as GPUs lidam bem com tamanhos grandes de batch, com o *warming up* da taxa de aprendizado (começando pequeno, aumentando). Se for ruim, usa batch menor. Para transformers como exceção eles usam batch pequeno
- Dica para compreender hiperparâmetros: [Smith, L. \(2018\)](#)

Problemas com treinamento profundo

Problema #1: DNN sofrem de gradientes instáveis; camadas diferentes podem aprender em velocidades bem diferentes. No retorno (backward) os gradientes podem reduzir (quase 0 nas camadas inferiores - **gradiente de fuga**) ou aumentar e ter atualização grande de pesos, divergindo (**explosão de gradiente**) - [Glorot e Bengio \(2010\)](#). Os neurônios usavam ativação regressão logística e inicialização de pesos aleatória com média 0 e desvio padrão 1

Possíveis soluções #1: manter variância IN e OUT equilibrada na rede. Inicializar com base no número de pesos de cada neurônio de entrada (fin) e o número de neurônios produzidos na camada (fout). Inicializações de Glorot e He. Acelera treinamento.

A inicialização Glorot/Xavier define a variância dos pesos como:

$$\text{Var}(W) = \frac{2}{f_{an_in} + f_{an_out}}$$

Ou o intervalo uniforme:

$$W \sim U \left[-\sqrt{\frac{6}{f_{an_in} + f_{an_out}}}, +\sqrt{\frac{6}{f_{an_in} + f_{an_out}}} \right]$$

Para Rede Linear(3,2)
 $W \sim U[-1.095, +1.095]$

Problemas com treinamento profundo

Possíveis soluções #1: Glorot usado em funções tanh, sigmoide, softmax. He usado em RELU e variantes com distribuição normal de variância $2/\text{fan_in}$. A inicialização LeCun (1990) é interessante para SELU, com distribuição normal, variância $1/\text{fan_in}$

Glorot é a padrão no Keras com dist. uniforme

```
keras.layers.Dense(10, activation='relu', kernel_initializer='he_normal') ou he_uniform  
he_avg_init = keras.initializers.VarianceScaling(scale=2., mode='fan_avg',  
distribution='uniform')  
keras.layers.Dense(10, activation='sigmoid', kernel_initializer=he_avg_init)
```

Conceito	Significado	Exemplo (Linear 3 → 2)
fan-in	nº de entradas por neurônio	3
fan-out	nº de neurônios de saída	2
Glorot	usa (fan-in + fan-out)	$\sqrt{6/5}$
He	usa só fan-in	$\sqrt{2/3}$

Para Rede Linear(3,2) He
 $W \sim U[-0.816, +0.816]$

Problemas com treinamento profundo

Problema #2: funções de ativação de não saturação

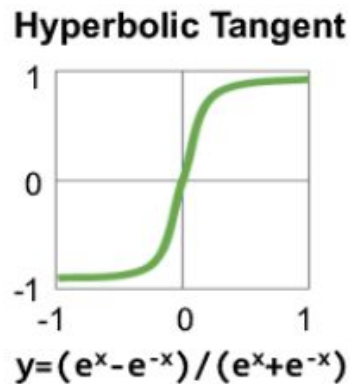
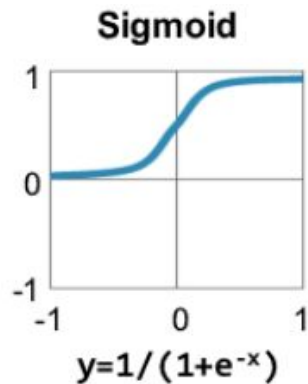
A função ReLU não satura valores positivos e é rápida para realizar cálculos. Contudo sofre do problema ReLUs agonizantes. Na prática os neurônios morrem e deixam de produzir saídas para qualquer coisa diferente de 0, sobretudo com taxa de aprendizagem alta. Ele morre quando a soma ponderada de suas entradas é negativa para todo conjunto de treinamento. Como o gradiente é 0 para entrada negativa, não atualiza.

Possíveis soluções #2: na LeakyReLU há um hiperparâmetro alpha (algo como 0.01) que é a inclinação para valores negativos e garante que as leaky ReLUs nunca morram. Podem entrar em coma profundo, mas em algum momento podem acordar. Ver [Bing Xu et al. \(2015\)](#). Já a RReLU (leaky RELU aleatória), alpha aleatório num intervalo durante o treino e fixado em valor médio durante o teste. Se comporta como um regularizador. Para conjuntos de imagens grandes, a ReLU paramétrica PReLU (o alpha é aprendido por backpropagation) **Para ir além, ver a [ELU](#) (exponential linear unit) e a [Scaled ELU](#) (SELU)**

```
keras.layers.Dense(10, kernel_initializer='he_normal'), keras.layers.LeakyReLU(alpha=0.2)  
Para SELU: activation='selu', kernel_initializer='lecun_normal')
```

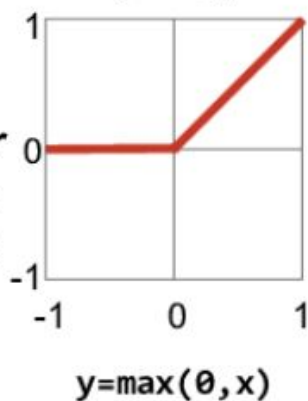

Problemas com treinamento profundo

Traditional
Non-Linear
Activation
Functions

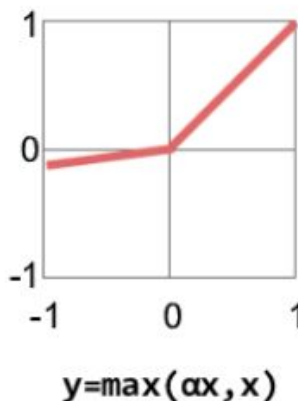


Modern
Non-Linear
Activation
Functions

Rectified Linear Unit (ReLU)

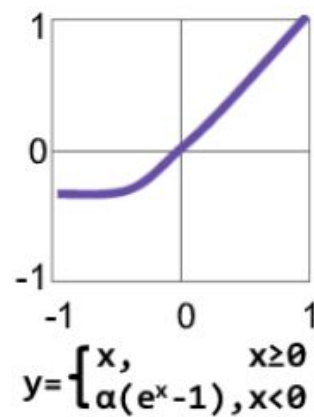


Leaky ReLU

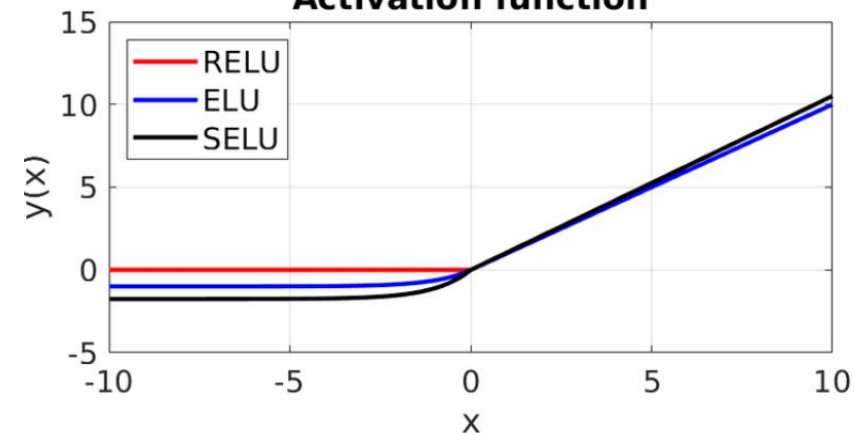


$\alpha = \text{small const. (e.g. 0.1)}$

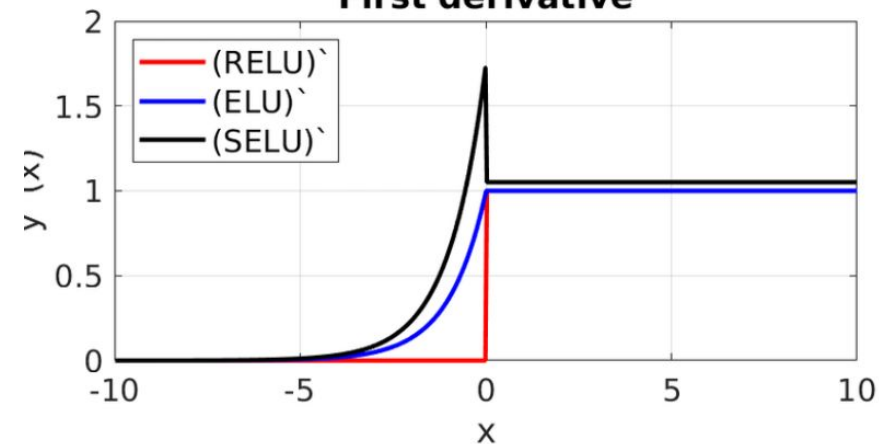
Exponential LU



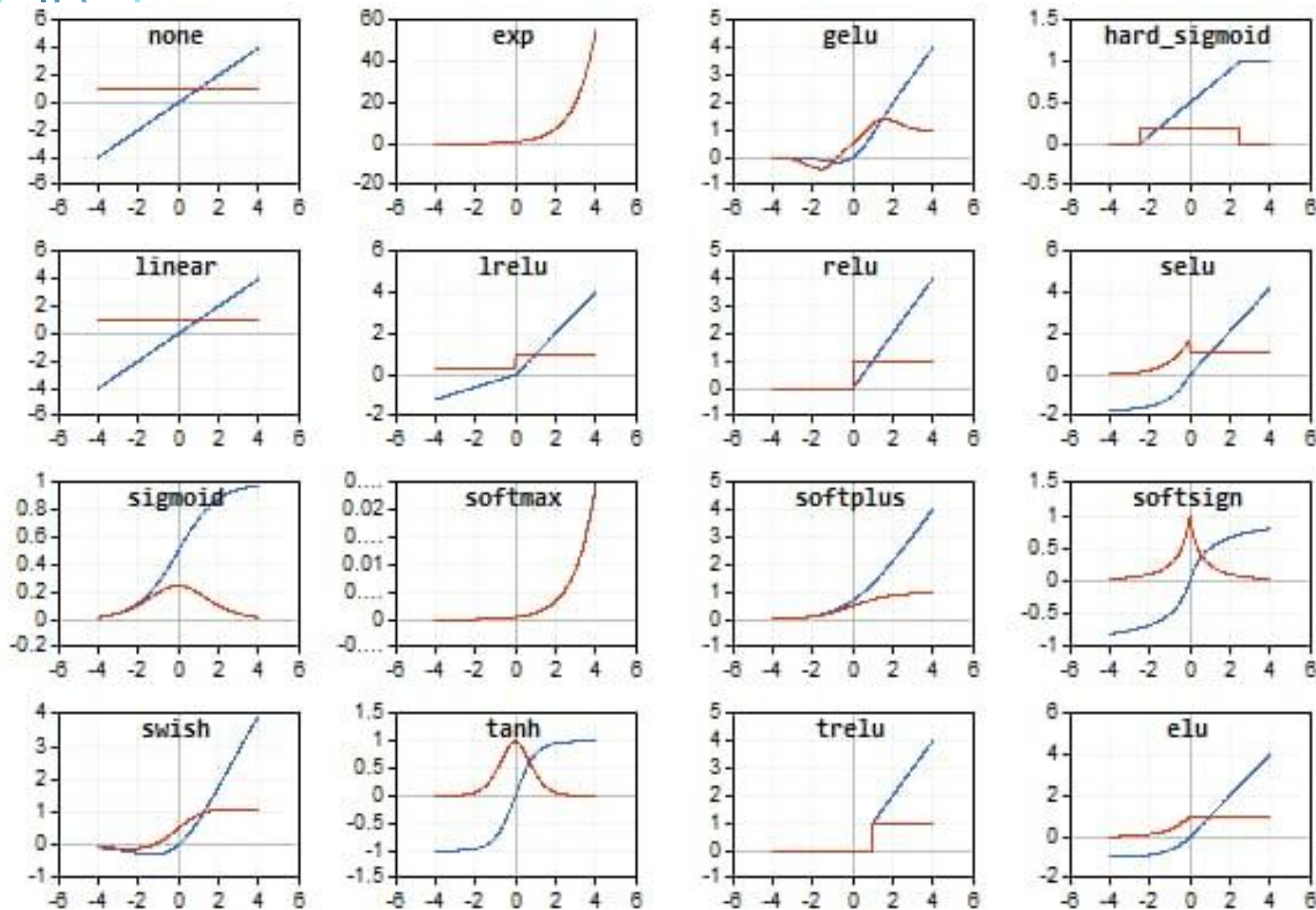
Activation function



First derivative



Problemas com treinamento profundo



azul - $f(x)$
vermelho - $df(x)/dx$

Problemas com treinamento profundo

Problema #3: normalização em batch (BatchNormalization) - [2015](#)

o uso de inicializações Glorot, He, Lecun, bem como funções não saturadas podem reduzir bastante o risco de fuga/explosão de gradientes no início do treino, mas não garante que os problemas não voltem ao longo do treino...

Possíveis soluções #3: o algoritmo avalia a média e o desvio-padrão da entrada em relação ao mini-batch atual, para centralizar em zero e normalizar as entradas, podendo ser incluído **antes** ou **depois** da função de ativação de cada camada oculta. Durante o treinamento vai calculando usando média móvel exponencial. Já os vetores de escalonamento e deslocamento são estimados por backpropagation.

Remediou a fuga de gradientes (mesmo com tanh e logística), ficou menos sensível à inicialização e usaram taxas de aprendizagem maiores.

O treino fica mais lento, mas converge mais rápido, menos épocas.

Contudo não é bom para RNN e batches pequenos

Problemas com treinamento profundo

```
model = keras.models.Sequential([ #batch depois da ativação
    keras.layers.Flatten(input_shape=[28,28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(300, activation='elu', kernel_initializer='he_normal'),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(100, activation='elu', kernel_initializer='he_normal'),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(10, activation='softmax')
])
```

```
model = keras.models.Sequential([ #batch antes da ativação
    keras.layers.Flatten(input_shape=[28,28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(300, use_bias=False, kernel_initializer='he_normal'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('elu'),
    keras.layers.Dense(100, use_bias=False, kernel_initializer='he_normal'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('elu'),
    keras.layers.Dense(10, activation='softmax')
])
```



Problemas com treinamento profundo

Tipo de modelo	Normalização padrão hoje
CNNs tradicionais	BatchNorm ainda forte
CNNs com batch pequeno	GroupNorm Keras
Transformers (texto, visão, multimodal)	LayerNorm / RMSNorm Keras
LLMs modernos (GPT, LLaMA, etc.)	RMSNorm
GANs recentes	LayerNorm / GroupNorm / InstanceNorm
Modelos sem normalização	Inicializações especiais

Problemas com treinamento profundo

A operação do RMSNorm é definida pela seguinte fórmula matemática:

$$\bar{\mathbf{x}} = \frac{\mathbf{x}}{\sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2 + \epsilon}}$$

A saída normalizada $\bar{\mathbf{x}}$ é calculada dividindo o vetor de entrada $\mathbf{x}$ pelo seu **Root Mean Square (RMS)**.

- $\mathbf{x}$ é o vetor de entrada para a camada (as ativações).
- D é a dimensão do vetor.
- $\sum_{i=1}^D x_i^2$ é a soma dos quadrados dos elementos de $\mathbf{x}$.
- $\sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2}$ é o cálculo do RMS (raiz quadrada da média dos quadrados).
- ϵ (epsilon) é uma pequena constante adicionada ao denominador para garantir **estabilidade numérica** e evitar a divisão por zero.

Problemas com treinamento profundo

⚡ RMSNorm vs. LayerNorm (Principal Diferença)

A diferença fundamental entre RMSNorm e LayerNorm reside no fato de o **RMSNorm não subtrair a média** (não centraliza as ativações).

Característica	RMSNorm	LayerNorm
Cálculo de Média	Não calcula nem subtrai a média.	Calcula e subtrai a média (μ).
Cálculo de Variância	Implícito no RMS.	Calcula o desvio-padrão (σ) a partir da variância.
Parâmetros de Aprendizagem	Apenas escala (γ).	Escala (γ) e deslocamento (β).
Complexidade Computacional	Mais rápido e exige menos operações.	Mais lento.

Problemas com treinamento profundo - LayerNorm

É uma operação que normaliza **cada amostra individualmente**, ao longo das **features**.

Ou seja, para uma entrada X do shape: (batch, features), o LayerNorm normaliza a **dimensão das features**, e não o batch. Isso estabiliza o fluxo de gradientes independentemente do tamanho do batch.

```
ln = nn.LayerNorm(normalized_shape=5)
```

Para um vetor de entrada $x = (x_1, x_2, \dots, x_F)$ da mesma amostra:

$$\mu = \frac{1}{F} \sum_{i=1}^F x_i$$

$$\sigma^2 = \frac{1}{F} \sum_{i=1}^F (x_i - \mu)^2$$

Então a normalização é:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

E depois vem a transformação aprendível:

$$y_i = \gamma \hat{x}_i + \beta$$

onde γ e β são parâmetros treináveis (como no BatchNorm).

Problemas com treinamento profundo

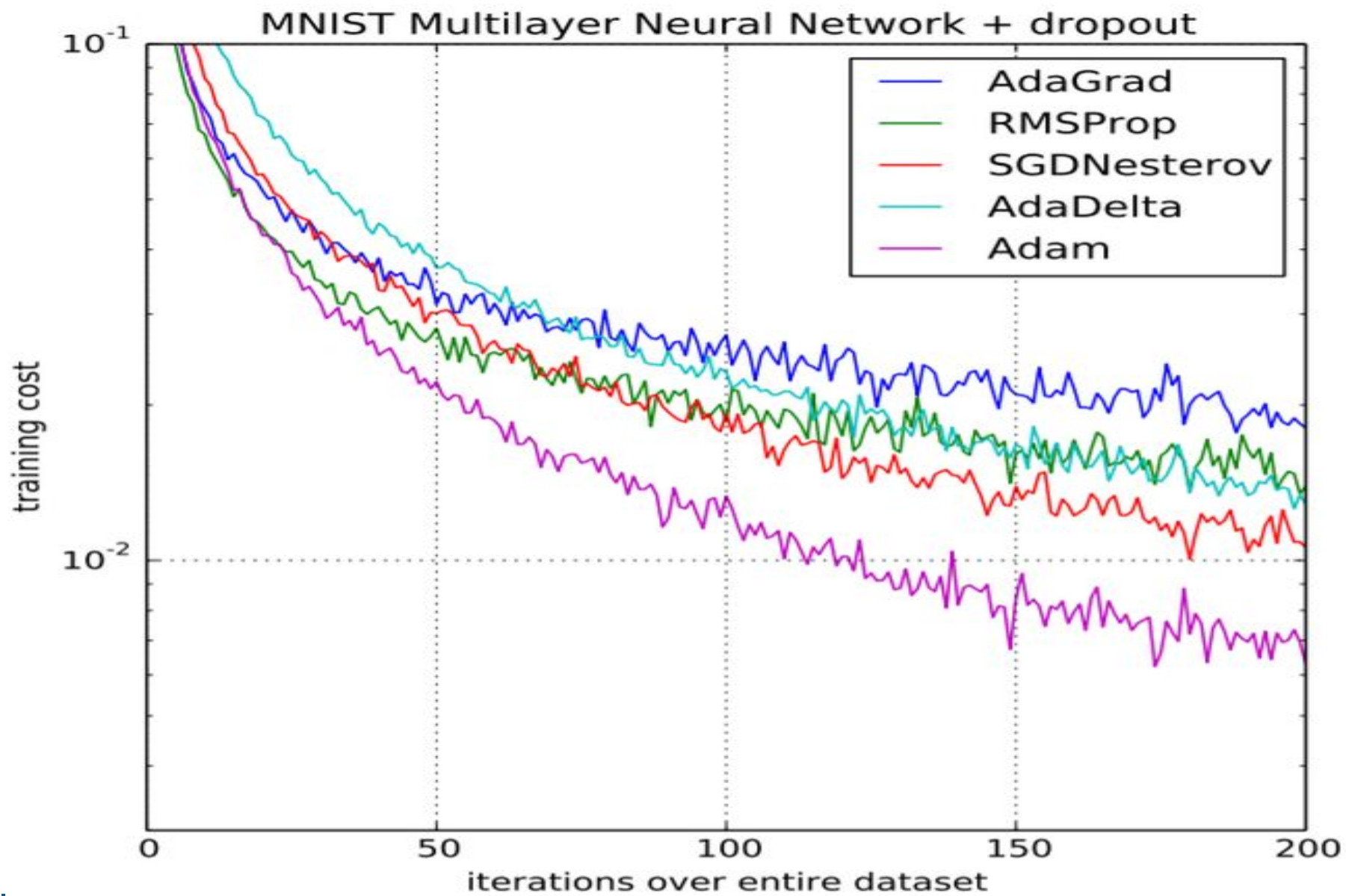
Problema #4: otimizador

o cálculo das derivadas parciais necessárias às atualizações de pesos continuam a ser realizadas inspiradas no backpropagation. Contudo, a expressão de atualização dos pesos apenas com gradiente descendente é limitada em termos de velocidade

Possíveis soluções #4: desde 1964 (Polyak, B.) se conhecem algoritmos mais rápidos em termos de convergência, usando conceitos de Momento (**momentum**), que usa também gradientes anteriores, para acelerar mais ou menos dependendo de onde está na superfície. Embora acrescente um hiperparâmetro, o “atrito”, o valor 0.9 é usado na maioria das aplicações. Nesterov (1983) fez pequena modificação, melhorando. Algoritmo NAG: `keras.optimizer.SGD(lr=0.001, momentum=0.9, nesterov=True)` Já o RMSProp (2012) adiciona termo de decaimento exponencial, dando maior peso aos gradientes mais recentes. O ADAM (2014) e variações (Nadam 2016) é um estimador adaptativo de momento, combinando ideias de momentum e RMSProp. Como em geral produzem modelos esparsos, pode-se usar em conjunto com L1 (lasso) para forçar zerar o maior número de pesos.

Otimizadores Mais Recentes e Alternativas ao Adam

- AdamW: (pytorch) O Adam original tem um problema com a **regularização de decaimento de peso** (weight decay). O AdamW "desacopla" o decaimento de peso do cálculo do gradiente adaptativo, o que geralmente leva a uma melhor generalização e resultados superiores
- Lion (EvoLved Sign Momentum): Um otimizador mais recente que é mais eficiente em termos de memória do que o Adam, pois rastreia apenas o momento. Ele usa a função de sinal dos gradientes para a atualização.
- Sophia (Second-order clipped isotropic P-proximation): Projetado especificamente para treinar modelos de linguagem grandes (LLMs), a Sophia aproxima derivadas de segunda ordem para uma convergência ainda mais rápida.
- RAdam (Rectified Adam): Este otimizador tenta resolver o problema de que o Adam pode convergir para mínimos ruins no início do treinamento, especialmente com tamanhos de lote pequenos, ajustando o termo de variância adaptativa.



Problemas com treinamento profundo

Problema #5: taxa de aprendizagem

a taxa fixa pode ser utilizada em contextos de parada antecipada (early stopping), mas em geral se usa algum cronograma de atualização, com base na fase de treinamento. Alta pode divergir. Baixa converge, mas demora muito

Early stopping é uma técnica de regularização que interrompe o treinamento quando o desempenho em um conjunto de validação não mais melhora. Monitora-se a perda de validação e quando não melhora em certo número de épocas (paciência), para e se usa o modelo deste ponto de parada..

Possíveis soluções #5:

- cronograma de potência: $n(t) = n_0 / (1+t/s)^c$. c normalmente é 1. n_0 e s são ajustáveis. A taxa cai a cada etapa. Após a etapa s , cai para a metade. Depois $/3$, depois $/4$...
- cronograma exponencial: $n(t) = n_0 \times 0.1^{(t/s)}$ - a cada s etapas, reduz 10x
- cronograma de desempenho: monitora validation loss e reduz taxa por fator λ
- 1cycle (2018): começa em n_0 , aumenta linearmente para n_1 na metade do treinamento. Depois cai para n_0 na segunda metade e vai caindo. O momentum também inicia 0.95, cai 0.85 e depois sobe para 0.95.

Problemas com treinamento profundo

```
optimizer = keras.optimizers.SGD(lr=0.01, decay=1e-4) # power scheduling, c=1, decay=1/s

#exponencial
def expotential_decay(lr0, s):
    def exponential_decay_fn(epoch):
        return lr0 * 0.1**(epoch/s)
    return expotential_decal_fn

exponential_decay_fn = expotential_decay(lr0=0.01,s=20)

lr_scheduler = keras.callbacks.LearningRateScheduler(exponential_decay_fn)
history = model.fit(X_train, y_train, [...], callbacks=[lr_scheduler])

#performance
lr_scheduler = keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5)
```

Problemas com treinamento profundo

Problema #6: sobreajuste

o early stopping e o batch normalization são também formas de regularização, existem outras técnicas como L1 (lasso), L2 (ridge), dropout e max-norm

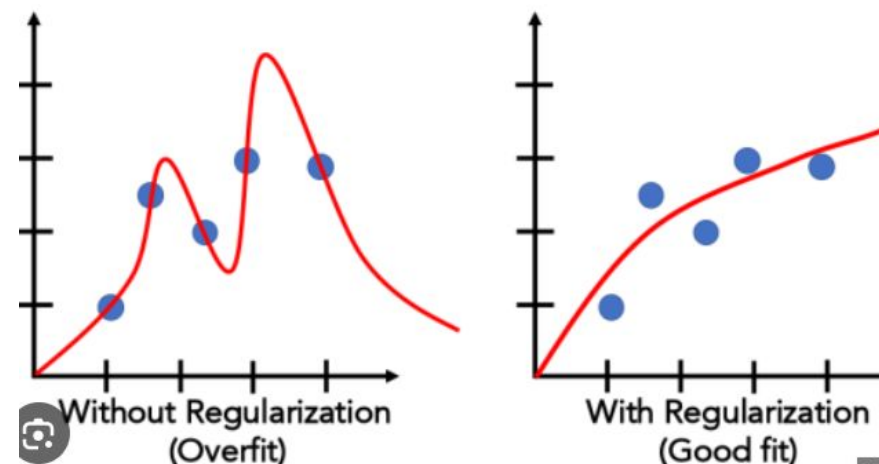
Possíveis soluções #6:

- L2 é usada para restringir os pesos de uma rede neural e L1 se deseja um modelo esparsos (com muitos pesos iguais a 0 - modelos de menos memória e com execução mais rápida)

```
#aplica a regularização a cada etapa, para calcular perda de regularização e soma à perda total  
layer = keras.layers.Dense(100, activation='elu', kernel_initializer='he_normal',  
                             kernel_regularizer=keras.regularizers.l2(0.01))
```

$$\text{RSS}_{\text{ridge}} = \sum_{i=1}^n [y_i - (\mathbf{w} \cdot \mathbf{x}_i + b)]^2 + \alpha \sum_{j=1}^p w_j^2$$

regularização ℓ_2



Problemas com treinamento profundo

Problema #6: sobreajuste

o early stopping e o batch normalization são também formas de regularização, existem outras técnicas como L1 (lasso), L2 (ridge), dropout e max-norm

#aplica a regularização em todas as camadas

```
from functools import partial
```

```
RegularizedDense = partial(keras.layers.Dense, activation='elu', kernel_initializer='he_normal',  
                           kernel_regularizer=keras.regularizers.l2(0.01))
```

```
model = keras.model.Sequential([  
    keras.layers.Flatten(input_shape=[28,28]),  
    RegularizedDense(300),  
    RegularizedDense(100),  
    RegularizedDense(10, activation='softmax', kernel_initializer='glorot_uniform')  
])
```

Problemas com treinamento profundo

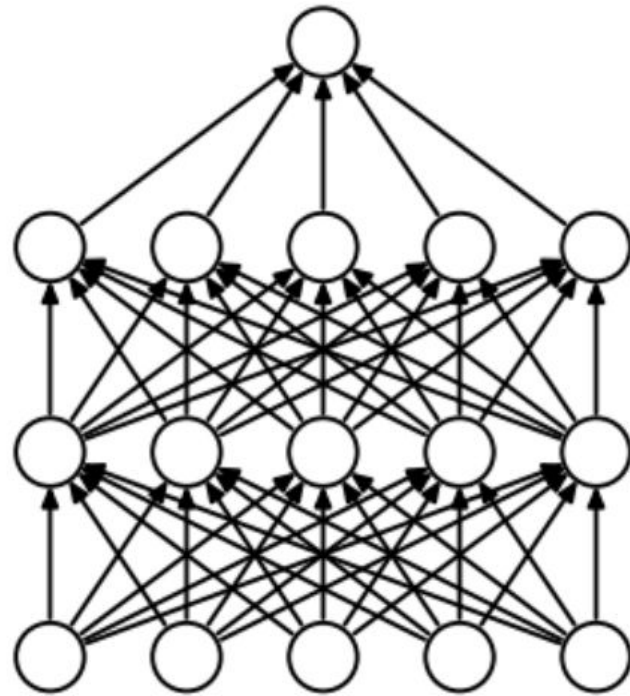
Problema #6: sobreajuste

outra solução é o DROPOUT, mais popular para DNN (Hinton et al., 2012 e Srivastava et al., 2014) - até redes mais modernas aumentam acurácia de 1 a 2% só colocando dropout. Se um modelo já tem 95% de acurácia, um aumento de 2% significa diminuir a taxa de erro em quase 40% (passando de 5% para 3%)

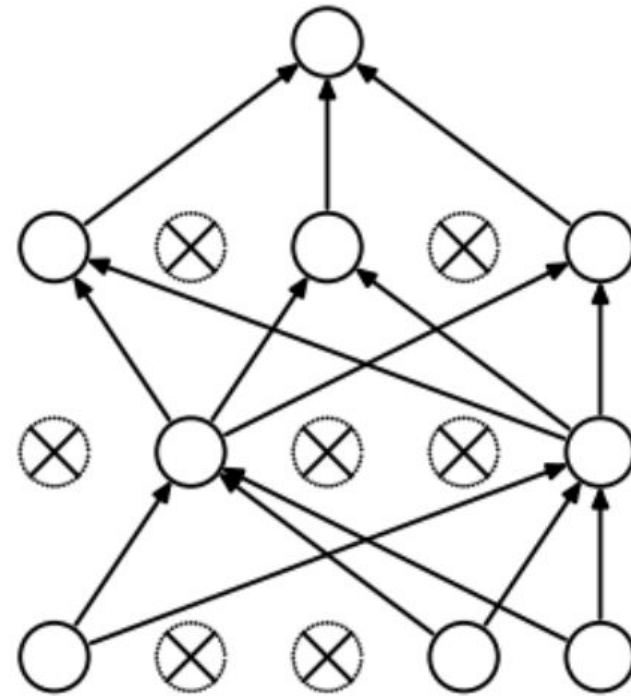
Possíveis soluções #6:

- p (taxa de dropout): 20 a 30% para RNN, 40 a 50% para CNN - a cada etapa todos os neurônios (com exceção dos de saída) tem probabilidade p de ser ignorado nesta etapa, podendo ser reativado na outra - os neurônios não conseguem se acomodar aos vizinhos; precisam ser os mais úteis possíveis por conta própria; não dependendo muito de suas entradas, ficando menos sensíveis e ficando mais atentos; obtém-se uma rede mais robusta que generaliza melhor
- na prática pode usar apenas nos neurônios de uma a três camadas superiores
- se perceber que está sobreajustando, tente aumentar dropout
- alguns modelos modernos usam dropout apenas após a última camada oculta. Ela pode desacelerar a convergência mas o modelo é bem melhor

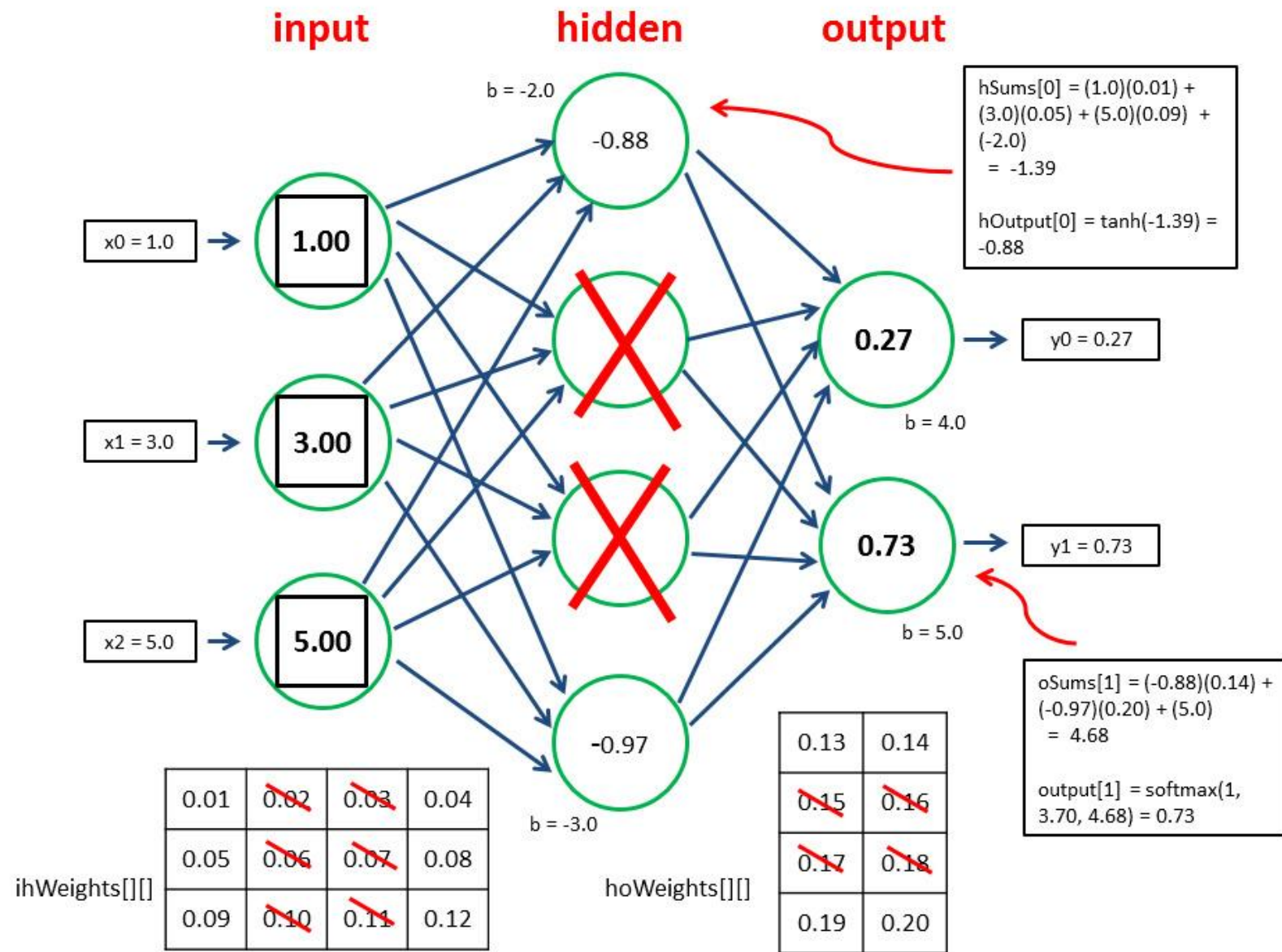
A cada etapa de treinamento é como se fosse uma rede diferente. Se tem-se N como o número total de neurônios descartáveis, tem-se 2^N redes. No final é como se tivéssemos uma “média dessas redes”

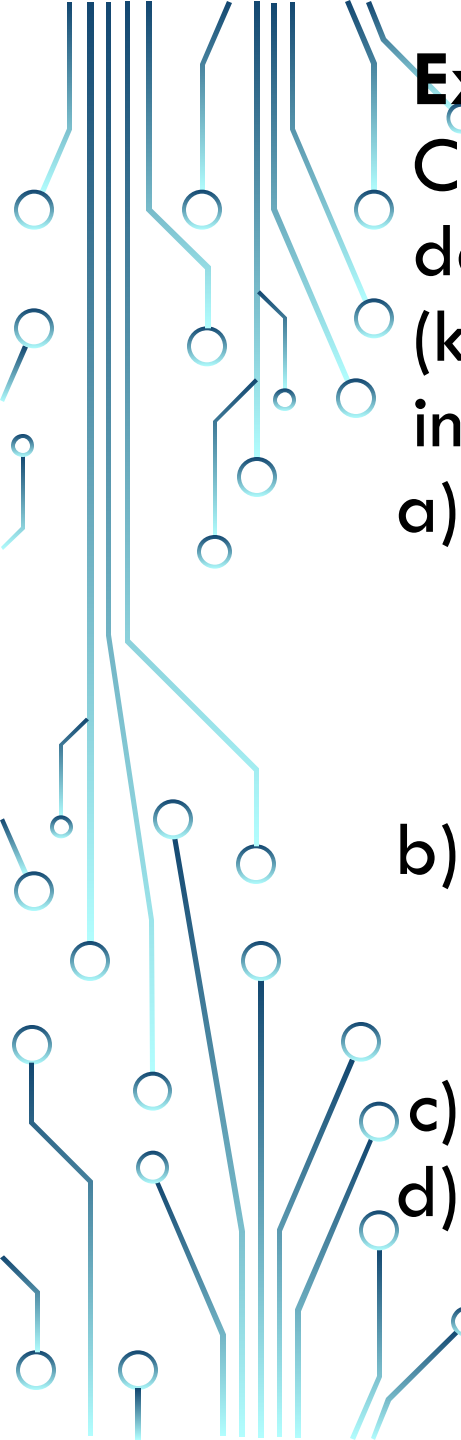


(a) Standard Neural Net



(b) After applying dropout.





Exercícios de fixação: (use seu framework preferido)

Com o dataset [CIFAR10](#), crie uma DNN com 20 camadas ocultas de 100 neurônios cada. Use a função de ativação ELU (keras.datasets ou torchvision.datasets). Como são 10 classes introduza softmax na camada de saída.

- a) usando a otimização NADAM e a EARLY STOPPING treine o modelo (dica: normalmente no NADAM $\backslash \text{beta_1}=0.9$, $\backslash \text{beta_2}=0.999$, $\backslash \text{epsilon}=1\text{e-}7$, com $\text{lr}=5\text{e-}5$ (fixo), com valor adequado de paciência ($\text{patience}=20$)
- b) agora rode com normalização em batch e compare as curvas de aprendizado. Está convergindo mais rápido? Produz um modelo melhor? Como afeta a velocidade do treinamento?
- c) acrescente o dropout
- d) treine mais uma vez usando o cronograma 1cycle e veja o que ocorre na velocidade de treinamento e na acurácia



Exercícios de fixação: (use seu framework preferido)
proposta de solução em keras neste [[link](#)], células
127-135; 138 com dropout normal e 143 em diante